

Note: The kernel version used is 4.8.0-rc3.

2. Type the following command to make the kernel source folder clean.

make ARCH=um mrproper

3. First, we need to generate a default configuration for the build. So, give the following command:

make ARCH=um defconfig

4. A `.config` file will be generated in the source directory. We need to modify the default kernel configuration and make sure certain configurations are enabled to make the build successful. To do this, execute the following command:

make ARCH=um menuconfig

5. You will see the menu shown in Figure 1 on the screen.
6. Go to the 'UML-specific' option and disable '64-bit kernel'.
7. From the main menu, go to 'General setup' → 'Initial RAM filesystem and RAM disk support' and select the option shown in Figure 3.
8. From the main menu, go to 'Kernel hacking' → 'Compile-time checks and compiler options'. Hack the kernel with debug information. Select the option shown in Figure 4.
9. Type the following command to start building the kernel:

make ARCH=um

Once the kernel is compiled, an executable named `Linux`, which can be executed as a user space program, will be created.

You can execute this Linux executable on the host Linux operating system. Since you can run this virtual Linux on an actual Linux OS, it is also called `Linux on Linux`. This virtual Linux uses the system call interface of the host operating system to work.

If you execute UML without any argument, you will notice two things.

The output of the execution is the same as the output when you boot the Linux OS on your machine. Some output may change but most of it will be the same.

Execution will fail because it requires a root filesystem to start with, which was not provided.

You will observe the same behaviour if you give no option or a bogus option to the 'root' variable while booting up, using `LILLO` or `GRUB`; note that the same problem in UML will just make the executable crash, but the host system will not be affected. So we will be able to learn how the kernel boots without the root file system, yet not cause any harm to the existing system. This is one of the many advantages of UML. You can do all the messy user and kernel programming,

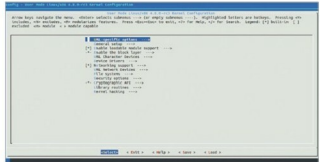


Figure 1: Linux kernel configuration

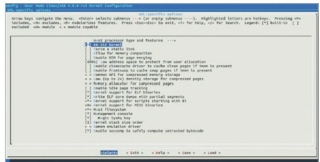


Figure 2: UML specific options

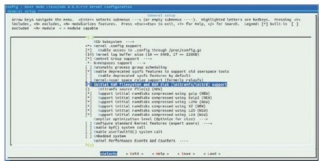


Figure 3: Initial RAM disk support

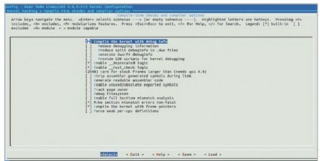


Figure 4: Compiling the kernel with debug information

change the boot parameters and even cause the kernel to crash — all without causing any harm to the existing system.

The root file system for this UML build is actually a file and not a physical device. Sample file system files can be downloaded from <http://fs.devloop.org.uk/>.

For example, let's use the Debian Wheezy root file system.